Лабораторна робота № 1–ЕС

Вивчення основних можливостей і базових команд середовища CLIPS

Мета роботи – вивчити особливості застосування середовища CLIPS для створення ЕС.

Теоретичні відомості

Середовище CLIPS призначене для побудови ЕС. Воно підтримує три основні способи подання знань:

– продукційні правила для подання евристичних, основаних на досвіді знань;

– функції для подання процедурних знань;

– об’єктно-орієнтоване програмування.

Середовище завантажується запуском файла clipswin.exe (рис. 1.1).

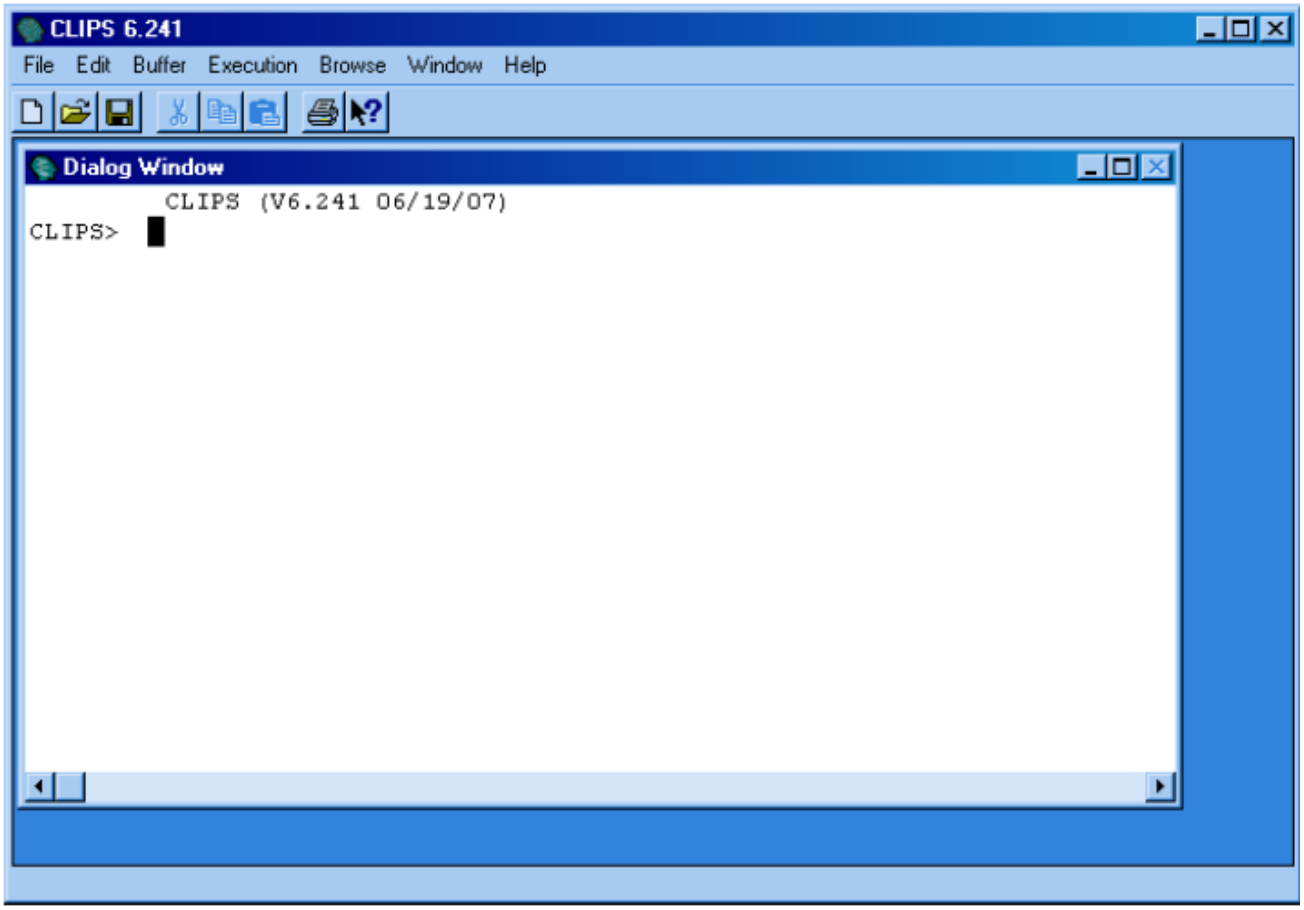

Рис. 1.1. Зовнішній вигляд оболонки мови CLIPS після запуску

Дочірнє вікно оболонки «Dialog Window», показане на рис. 1.1, використовується для виконання команд CLIPS і отримання результату. Це – свого роду командний рядок оболонки, де можна виконати команду

CLIPS, записавши її і натиснувши клавішу <Enter>. Наприклад, написавши в рядку код (включаючи дужки)

(printout t "Hello world" crlf)

і натиснувши <Enter>, ми змусимо CLIPS виконати функцію виводу на екран і отримаємо рядок «Hello world» відразу після нашого виклику, після чого оболонка повернеться в режим очікування. Для скидання середовища CLIPS у початковий стан використовується команда (clear) або відповідний пункт меню Execution.

Для того щоб почати писати програму на мові CLIPS, потрібно створити новий файл шляхом вибору пункту меню File – New (рис. 1.2).

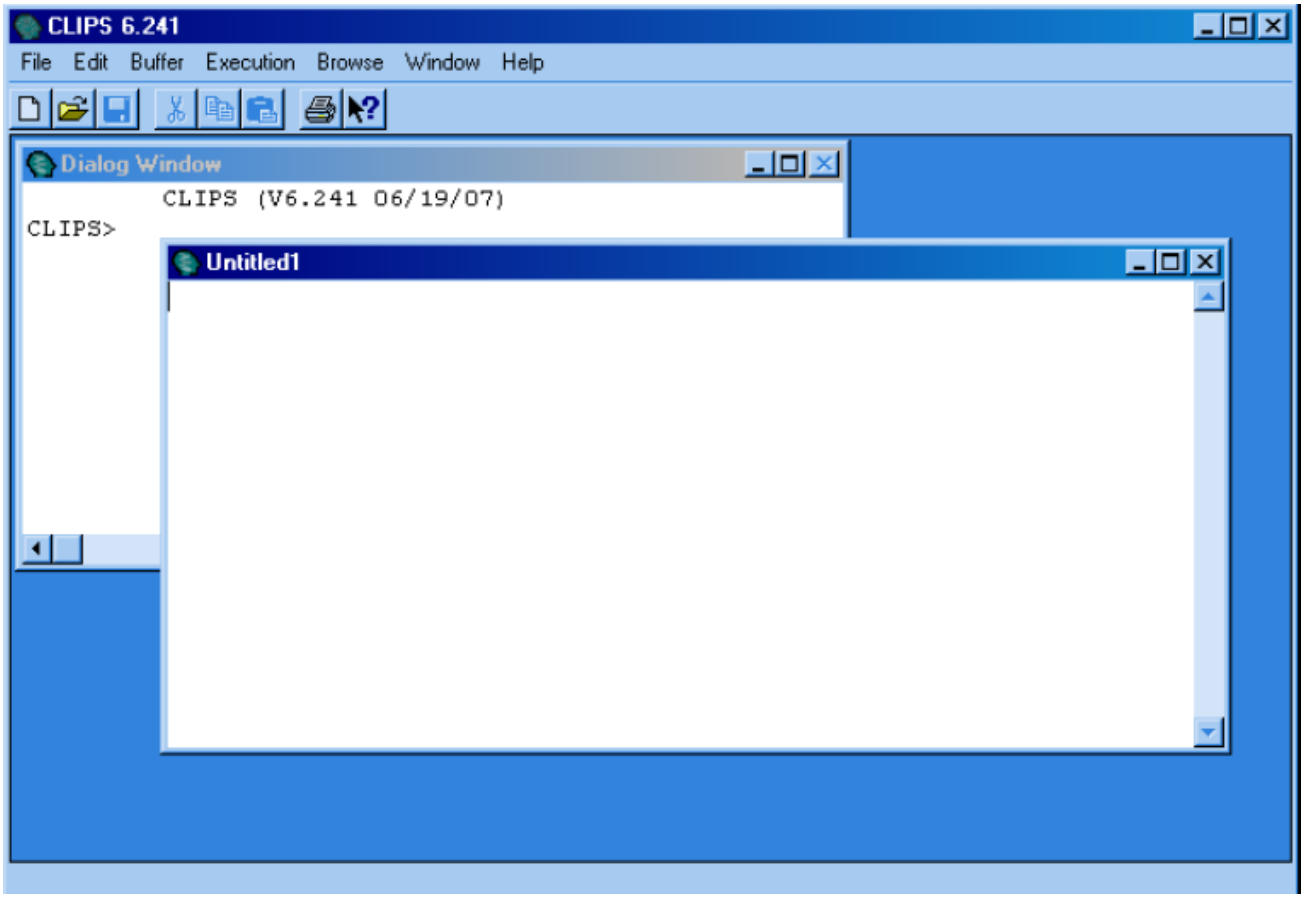

Рис. 1.2. Вікно для складання CLIPS-програм

Для того щоб команди, написані у вікні, були виконані, необхідно виділити їх і натиснути комбінацію Ctrl-M або вибрати пункт меню Buffer – Batch Selection.

Відразу зазначимо, що програма на CLIPS принципово відрізняється від програми на класичній мові програмування, такій, як, наприклад, C або Pascal. Запуск програми на CLIPS фактично означає запуск механізму логічного висновку, основаного на правилах, що є у системі, які у свою чергу ґрунтуються на фактах. У загальному випадку

найпростіша програма на CLIPS складається з набору правил і функцій, що маніпулюють фактами, які розглянемо трохи пізніше.

Примітивні типи даних мови CLIPS. Примітивними типами даних мови CLIPS є: *float, integer, symbol, string, external-address, fact-address, instance-name, instance-address*. Для зберігання числової інформації призначаються типи float і integer, для символьної – symbol і string.

Число в CLIPS може складатися тільки з символів цифр (0 – 9), десяткової крапки (.), знака (+ або –) і експонентного символу (е) з відповідним знаком у разі подання числа в експоненційній формі. Нижче наведено приклади допустимих у CLIPS уявлень цілих і речових типів:

Цілі: 237; 15; +12; –32

Речові: 237e3; 15.09; +12;–32.3e–7

Значенням типу symbol може бути будь-яка послідовність символів, що починається з будь-якого некеруючого ASCII-символу. Значення типу symbol закінчується обмежувачем. Обмежувачами є будь-які символи, що не відображуються (наприклад, пробіл, символ табуляції або переходу на інший рядок), подвійні лапки, які відкриває або закриває кругла дужка, символи &, |, <і ~. Крапка з комою (;) є символом початку коментарів і також може обмежувати значення типу symbol. Нижче наведено зразки допустимих значень типу symbol:

good Hello B76–HI bad_value

127A 456–93–039 @ + = –% 2each

У мові CLIPS існують зарезервовані слова, які не можуть бути використані як значення типу symbol. Список зарезервованих слів досить великий, його можна знайти в документації з мови.

Значення типу string являє собою рядок символів, узятих у подвійні лапки. Приклади допустимих значень string наведено нижче:

"Life is good" "A and B" "1 number" "Value"

У рамках цієї лабораторної роботи інші типи розглядатися не будуть. Для їх вивчення необхідно звернутися до документації і відповідної літератури.

Особливості виклику функцій в мові CLIPS. *Функцією* у CLIPS називається частина коду, що має ім'я і повертає корисний результат або виконує корисні дії (наприклад, відображення інформації на екрані). Функції, які не повертають результат і виконують деяку корисну роботу, зазвичай називаються командами.

CLIPS оперує кількома типами функцій – зовнішні функції, системні функції, призначені для користувача функції, родові функції. Зовнішні

функції можуть створюватися на інших мовах програмування (наприклад, C) і потім підключаються до CLIPS на етапі компілювання або функціонування середовища. Системні функції створені розробниками середовища CLIPS і описані в керівництві користувача. Функції, які задає користувач, визначаються у середовищі CLIPS за допомогою спеціального конструктора ***deffunction*.**

Зверніть увагу на такі особливості виклику функцій у CLIPS:

- ім'я функції разом з усіма її аргументами береться у круглі дужки;
- аргументи функції завжди слідують після імені функції;
- аргументи відокремлюються один від одного принаймні одним пробілом;
- аргументами функцій можуть бути змінні примітивних типів, константи або виклики інших функцій.

Нижче наведено приклади використання функцій + (арифметичне додавання) і * (арифметичне множення):

(+ 3 4 5)
(5 6.0 2)*
(+ 3 (8 9) 4)*

Ви можете побачити результат виконання функцій, увівши їх у діалогове вікно оболонки CLIPS (див. рис. 1.1).

***Факти в мові CLIPS*.** *Факт* – одна з основних форм подання даних у CLIPS. Кожен факт – це певний набір даних, що зберігається у поточному списку фактів, тобто робочій пам'яті системи. Список фактів являє собою універсальне сховище фактів і є частиною бази знань. Обсяг списку фактів обмежений розміром пам'яті комп'ютера.

У системі CLIPS фактом є список неподільних (або атомарних) значень примітивних типів даних, узятих у дужки. Наприклад,

(life is good)
(power failed 22 minutes)
(temperature 10)
(model "OPEL Omega")

є фактами мови CLIPS. Зазвичай факти використовуються для того, щоб внести в систему заздалегідь відомі знання або додати інформацію, отриману в процесі діалогу з користувачем або в результаті обчислень.

Факти бувають *простими* і *складовими*. Складові факти є аналогами структур у класичних мовах програмування і визначаються за допомогою спеціальних конструкторів. Розглянемо функції для роботи з простими фактами:

Функція assert – додавання факту в список. Функція **assert** приймає як параметри послідовність фактів, які підлягають додаванню у список фактів. Синтаксичну структуру виклику функції **assert** можна подати таким чином:

(assert <факт> <факт> <факт>)

Для того щоб мати можливість спостерігати процеси додавання, видалення або зміни фактів, необхідно викликати пункт меню Execution – Watch і встановити прапорець напроти пункту «Facts» (рис. 1.3).

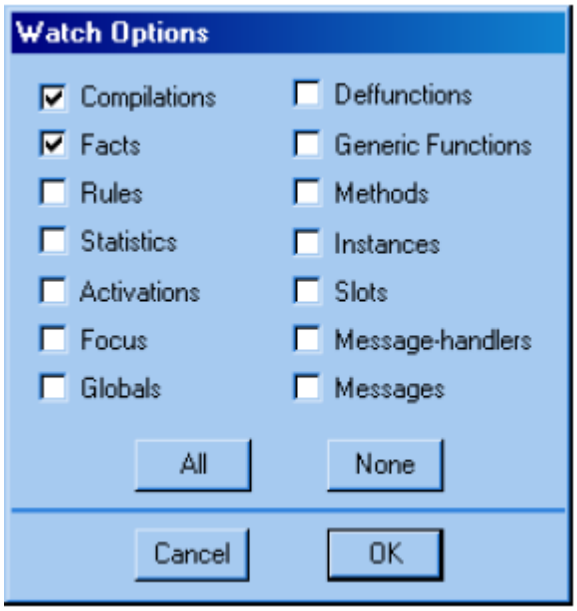

Рис. 1.3. Установка параметрів трасування

Розглянемо приклад використання функції assert. Перейдіть у діалогове вікно CLIPS (див. рис. 1.1) і введіть такий вираз:

(assert (weather is fine))

Будьте уважні з дужками. У тому випадку якщо Ви правильно ввели команду, у список фактів повинен додатися факт (weather is fine), після чого з'явиться результат виконання

==> f-0 (weather is fine)
<Fact-0>

це означає, що факт був доданий у систему і отримав номер 0. При успішному виконанні функція assert повертає адресу останнього доданого факту. Якщо під час додавання деякого факту сталася помилка, команда припиняє свою роботу і повертає значення FALSE.

Перевірити поточний стан списку фактів можна або за допомогою введення в діалоговому вікні команди (facts) або викликавши пункт меню Window – 1 Fact (рис. 1.4). У другому випадку відкриється вікно зі списком фактів.

При додаванні факту можна використовувати виклики функцій. Наприклад, виконання команди

(assert (totalcost (10 13))*

викличе функцію множення і її результат запишеться як поле факту. При цьому в пам'ять додасться факт (totalcost 130).

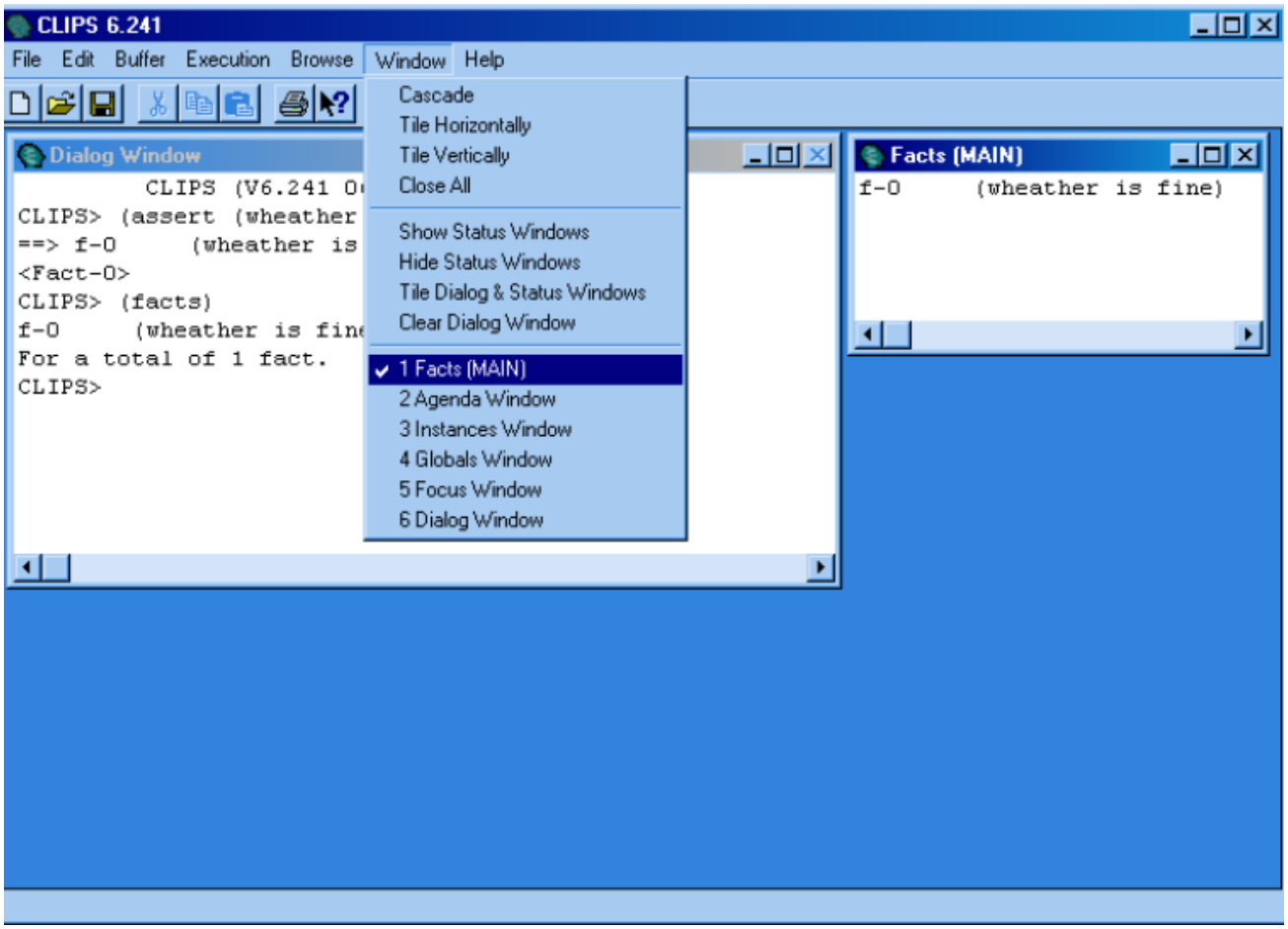

Рис. 1.4. Виклик вікна зі списком фактів

Функція ***retract*** – видалення фактів. Для видалення фактів з поточного списку фактів у системі CLIPS передбачено функцію **retract.** Кожним викликом цієї функції можна видалити довільну кількість фактів. У разі якщо був включений режим перегляду зміни списку фактів, то відповідне інформаційне повідомлення буде відображатися у вікні CLIPS при видаленні кожного факту.

Синтаксичну структуру виклику функції **retract** можна подати таким чином:

(retract <визначення-факту> <визначення-факту> ...)

або

*(retract *)*

Аргумент <*визначення-факту*> може бути або змінною, пов'язаною з адресою факту (адреса факту повертається командою assert), або індексом факту без префікса (наприклад, 3 для факту з індексом f-3), або виразом, який обчислює цей індекс (наприклад, (+ 1 2) для факту з індексом f-3). Якщо як аргумент функції **retract** використовувався символ * (зірочка), то з поточного списку будуть вилучені всі факти. Функція **retract** не має значення, що повертається.

Розглянемо роботу функції **retract** на прикладі. Перейдіть у діалогове вікно CLIPS (див. рис. 1.1) і введіть такі команди:

(clear)
(assert (a) (b) (c) (d) (e) (f))
(retract 0 (+ 0 2) (+ 0 2 2))

Результатом виконання наведеної вище послідовності команд буде:

1) очищення списку фактів;
2) додавання в систему шести фактів;
3) видалення з системи фактів з номерами 0, 2 і 4.

Зверніть увагу на те, як міняється список фактів у вікні Facts з кожною виконаною командою. При успішному виконанні зазначених команд у списку залишаться факти (b), (d) і (f).

Порядок виконання роботи

1. Виконайте самостійно такий набір команд:

(clear)
(assert (color red))
(assert (color blue) (value 3 4)))
(assert(suggest "Turn on the left"))

2. Виведіть список фактів у діалогове вікно, переконайтеся у тому, що всі факти додано правильно. Зверніть увагу на факт Value.

3. Самостійно придумайте і додайте факт, який означає, що лампу в коридорі увімкнено.

4.Самостійно придумайте і додайте будь-яку послідовність фактів не менше 5 і не більше 10. Видаліть вибірково кілька фактів, при цьому обов'язково застосувати в одному з варіантів функцію додавання.

5. Дано цілі числа 5, 9, 20, 1. Виконайте з ними прості арифметичні дії, використовуючи функції додавання, віднімання, ділення і множення. Результат виконання виведіть у діалоговому вікні.

Звіт про виконання роботи

Звіт про виконання лабораторної роботи № 1–ЕС має містити:

1. Титульний аркуш.
2. Програмний код, що складається з необхідного набору команд і функцій, а також результатів їх виконання для кожного пункту даної лабораторної роботи.
3. Екранні форми, що показують результати виконання кожного пункту даної лабораторної роботи.
4. Висновки.